

4.2 The Seven Functions Used in This Book

In this section, we will briefly discuss the seven most important functions used in the analysis of algorithms. We will use only these seven simple functions for almost all the analysis we do in this book. In fact, a section that uses a function other than one of these seven will be marked with a star (★) to indicate that it is optional. In addition to these seven fundamental functions, an appendix (available on the companion website) contains a list of other useful mathematical facts that apply in the analysis of data structures and algorithms.

The Constant Function

The simplest function we can think of is the **constant function**, that is,

$$f(n) = c,$$

for some fixed constant c , such as $c = 5$, $c = 27$, or $c = 2^{10}$. That is, for any argument n , the constant function $f(n)$ assigns the value c . In other words, it does not matter what the value of n is; $f(n)$ will always be equal to the constant value c .

Because we are most interested in integer functions, the most fundamental constant function is $g(n) = 1$, and this is the typical constant function we use in this book. Note that any other constant function, $f(n) = c$, can be written as a constant c times $g(n)$. That is, $f(n) = cg(n)$ in this case.

As simple as it is, the constant function is useful in algorithm analysis because it characterizes the number of steps needed to do a basic operation on a computer, like adding two numbers, assigning a value to a variable, or comparing two numbers.

The Logarithm Function

One of the interesting and sometimes even surprising aspects of the analysis of data structures and algorithms is the ubiquitous presence of the **logarithm function**, $f(n) = \log_b n$, for some constant $b > 1$. This function is defined as the inverse of a power, as follows:

$$x = \log_b n \text{ if and only if } b^x = n.$$

The value b is known as the **base** of the logarithm. Note that by the above definition, for any base $b > 0$, we have that $\log_b 1 = 0$.

The most common base for the logarithm function in computer science is 2 as computers store integers in binary. In fact, this base is so common that we will typically omit it from the notation when it is 2. That is, for us,

$$\log n = \log_2 n.$$

We note that most handheld calculators have a button marked LOG, but this is typically for calculating the logarithm base-10, not base-two.

Computing the logarithm function exactly for any integer n involves the use of calculus, but we can use an approximation that is good enough for our purposes without calculus. We recall that the **ceiling** of a real number, x , is the smallest integer greater than or equal to x , denoted with $\lceil x \rceil$. The ceiling of x can be viewed as an integer approximation of x since we have $x \leq \lceil x \rceil < x + 1$. For a positive integer, n , we repeatedly divide n by b and stop when we get a number less than or equal to 1. The number of divisions performed is equal to $\lceil \log_b n \rceil$. We give below three examples of the computation of $\lceil \log_b n \rceil$ by repeated divisions:

- $\lceil \log_3 27 \rceil = 3$, because $((27/3)/3)/3 = 1$;
- $\lceil \log_4 64 \rceil = 3$, because $((64/4)/4)/4 = 1$;
- $\lceil \log_2 12 \rceil = 4$, because $((12/2)/2)/2)/2 = 0.75 \leq 1$.

The following proposition describes several important identities that involve logarithms for any base greater than 1.

Proposition 4.1 (Logarithm Rules): *Given real numbers $a > 0$, $b > 1$, $c > 0$, and $d > 1$, we have:*

1. $\log_b(ac) = \log_b a + \log_b c$
2. $\log_b(a/c) = \log_b a - \log_b c$
3. $\log_b(a^c) = c \log_b a$
4. $\log_b a = \log_d a / \log_d b$
5. $b^{\log_d a} = a^{\log_d b}$

By convention, the unparenthesized notation $\log n^c$ denotes the value $\log(n^c)$. We use a notational shorthand, $\log^c n$, to denote the quantity, $(\log n)^c$, in which the result of the logarithm is raised to a power.

The above identities can be derived from converse rules for exponentiation that we will present on page 161. We illustrate these identities with a few examples.

Example 4.2: *We demonstrate below some interesting applications of the logarithm rules from Proposition 4.1 (using the usual convention that the base of a logarithm is 2 if it is omitted).*

- $\log(2n) = \log 2 + \log n = 1 + \log n$, by rule 1
- $\log(n/2) = \log n - \log 2 = \log n - 1$, by rule 2
- $\log n^3 = 3 \log n$, by rule 3
- $\log 2^n = n \log 2 = n \cdot 1 = n$, by rule 3
- $\log_4 n = (\log n) / \log 4 = (\log n) / 2$, by rule 4
- $2^{\log n} = n^{\log 2} = n^1 = n$, by rule 5.

As a practical matter, we note that rule 4 gives us a way to compute the base-two logarithm on a calculator that has a base-10 logarithm button, LOG, for

$$\log_2 n = \text{LOG } n / \text{LOG } 2.$$

The Linear Function

Another simple yet important function is the *linear function*,

$$f(n) = n.$$

That is, given an input value n , the linear function f assigns the value n itself.

This function arises in algorithm analysis any time we have to do a single basic operation for each of n elements. For example, comparing a number x to each element of an array of size n will require n comparisons. The linear function also represents the best running time we can hope to achieve for any algorithm that processes each of n objects that are not already in the computer's memory, because reading in the n objects already requires n operations.

The N -Log- N Function

The next function we discuss in this section is the *n -log- n function*,

$$f(n) = n \log n,$$

that is, the function that assigns to an input n the value of n times the logarithm base-two of n . This function grows a little more rapidly than the linear function and a lot less rapidly than the quadratic function; therefore, we would greatly prefer an algorithm with a running time that is proportional to $n \log n$, than one with quadratic running time. We will see several important algorithms that exhibit a running time proportional to the n -log- n function. For example, the fastest possible algorithms for sorting n arbitrary values require time proportional to $n \log n$.

The Quadratic Function

Another function that appears often in algorithm analysis is the *quadratic function*,

$$f(n) = n^2.$$

That is, given an input value n , the function f assigns the product of n with itself (in other words, " n squared").

The main reason why the quadratic function appears in the analysis of algorithms is that there are many algorithms that have nested loops, where the inner loop performs a linear number of operations and the outer loop is performed a linear number of times. Thus, in such cases, the algorithm performs $n \cdot n = n^2$ operations.

Nested Loops and the Quadratic Function

The quadratic function can also arise in the context of nested loops where the first iteration of a loop uses one operation, the second uses two operations, the third uses three operations, and so on. That is, the number of operations is

$$1 + 2 + 3 + \cdots + (n-2) + (n-1) + n.$$

In other words, this is the total number of operations that will be performed by the nested loop if the number of operations performed inside the loop increases by one with each iteration of the outer loop. This quantity also has an interesting history.

In 1787, a German schoolteacher decided to keep his 9- and 10-year-old pupils occupied by adding up the integers from 1 to 100. But almost immediately one of the children claimed to have the answer! The teacher was suspicious, for the student had only the answer on his slate. But the answer, 5050, was correct and the student, Carl Gauss, grew up to be one of the greatest mathematicians of his time. We presume that young Gauss used the following identity.

Proposition 4.3: For any integer $n \geq 1$, we have:

$$1 + 2 + 3 + \cdots + (n-2) + (n-1) + n = \frac{n(n+1)}{2}.$$

We give two “visual” justifications of Proposition 4.3 in Figure 4.3.

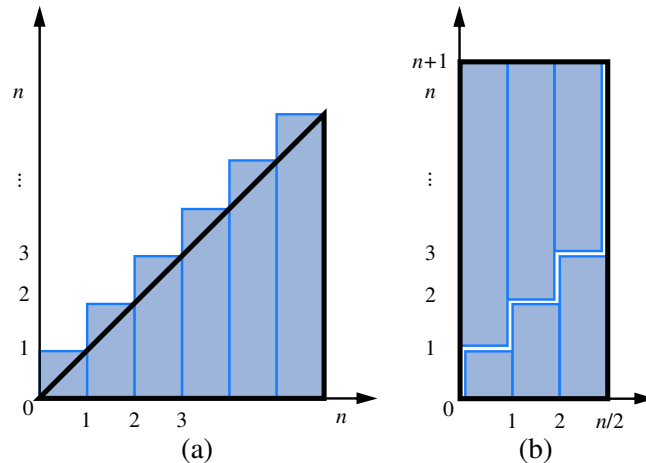


Figure 4.3: Visual justifications of Proposition 4.3. Both illustrations visualize the identity in terms of the total area covered by n unit-width rectangles with heights $1, 2, \dots, n$. In (a), the rectangles are shown to cover a big triangle of area $n^2/2$ (base n and height n) plus n small triangles of area $1/2$ each (base 1 and height 1). In (b), which applies only when n is even, the rectangles are shown to cover a big rectangle of base $n/2$ and height $n+1$.

The lesson to be learned from Proposition 4.3 is that if we perform an algorithm with nested loops such that the operations in the inner loop increase by one each time, then the total number of operations is quadratic in the number of times, n , we perform the outer loop. To be fair, the number of operations is $n^2/2 + n/2$, and so this is just over half the number of operations than an algorithm that uses n operations each time the inner loop is performed. But the order of growth is still quadratic in n .

The Cubic Function and Other Polynomials

Continuing our discussion of functions that are powers of the input, we consider the **cubic function**,

$$f(n) = n^3,$$

which assigns to an input value n the product of n with itself three times.

The cubic function appears less frequently in the context of algorithm analysis than the constant, linear, and quadratic functions previously mentioned, but it does appear from time to time.

Polynomials

The linear, quadratic and cubic functions can each be viewed as being part of a larger class of functions, the **polynomials**. A **polynomial** function has the form,

$$f(n) = a_0 + a_1n + a_2n^2 + a_3n^3 + \cdots + a_dn^d,$$

where $a_0, a_1, \dots, a_d$ are constants, called the **coefficients** of the polynomial, and $a_d \neq 0$. Integer d , which indicates the highest power in the polynomial, is called the **degree** of the polynomial.

For example, the following functions are all polynomials:

- $f(n) = 2 + 5n + n^2$
- $f(n) = 1 + n^3$
- $f(n) = 1$
- $f(n) = n$
- $f(n) = n^2$

Therefore, we could argue that this book presents just four important functions used in algorithm analysis, but we will stick to saying that there are seven, since the constant, linear, and quadratic functions are too important to be lumped in with other polynomials. Running times that are polynomials with small degree are generally better than polynomial running times with larger degree.

Summations

A notation that appears again and again in the analysis of data structures and algorithms is the **summation**, which is defined as follows:

$$\sum_{i=a}^b f(i) = f(a) + f(a+1) + f(a+2) + \cdots + f(b),$$

where a and b are integers and $a \leq b$. Summations arise in data structure and algorithm analysis because the running times of loops naturally give rise to summations.

Using a summation, we can rewrite the formula of Proposition 4.3 as

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}.$$

Likewise, we can write a polynomial $f(n)$ of degree d with coefficients $a_0, \dots, a_d$ as

$$f(n) = \sum_{i=0}^d a_i n^i.$$

Thus, the summation notation gives us a shorthand way of expressing sums of increasing terms that have a regular structure.

The Exponential Function

Another function used in the analysis of algorithms is the **exponential function**,

$$f(n) = b^n,$$

where b is a positive constant, called the **base**, and the argument n is the **exponent**. That is, function $f(n)$ assigns to the input argument n the value obtained by multiplying the base b by itself n times. As was the case with the logarithm function, the most common base for the exponential function in algorithm analysis is $b = 2$. For example, an integer word containing n bits can represent all the nonnegative integers less than 2^n . If we have a loop that starts by performing one operation and then doubles the number of operations performed with each iteration, then the number of operations performed in the n^{th} iteration is 2^n .

We sometimes have other exponents besides n , however; hence, it is useful for us to know a few handy rules for working with exponents. In particular, the following **exponent rules** are quite helpful.

Proposition 4.4 (Exponent Rules): Given positive integers a , b , and c , we have

1. $(b^a)^c = b^{ac}$
2. $b^a b^c = b^{a+c}$
3. $b^a / b^c = b^{a-c}$

For example, we have the following:

- $256 = 16^2 = (2^4)^2 = 2^{4 \cdot 2} = 2^8 = 256$ (Exponent Rule 1)
- $243 = 3^5 = 3^{2+3} = 3^2 3^3 = 9 \cdot 27 = 243$ (Exponent Rule 2)
- $16 = 1024/64 = 2^{10}/2^6 = 2^{10-6} = 2^4 = 16$ (Exponent Rule 3)

We can extend the exponential function to exponents that are fractions or real numbers and to negative exponents, as follows. Given a positive integer k , we define $b^{1/k}$ to be k^{th} root of b , that is, the number r such that $r^k = b$. For example, $25^{1/2} = 5$, since $5^2 = 25$. Likewise, $27^{1/3} = 3$ and $16^{1/4} = 2$. This approach allows us to define any power whose exponent can be expressed as a fraction, for $b^{a/c} = (b^a)^{1/c}$, by Exponent Rule 1. For example, $9^{3/2} = (9^3)^{1/2} = 729^{1/2} = 27$. Thus, $b^{a/c}$ is really just the c^{th} root of the integral exponent b^a .

We can further extend the exponential function to define b^x for any real number x , by computing a series of numbers of the form $b^{a/c}$ for fractions a/c that get progressively closer and closer to x . Any real number x can be approximated arbitrarily closely by a fraction a/c ; hence, we can use the fraction a/c as the exponent of b to get arbitrarily close to b^x . For example, the number 2^π is well defined. Finally, given a negative exponent d , we define $b^d = 1/b^{-d}$, which corresponds to applying Exponent Rule 3 with $a = 0$ and $c = -d$. For example, $2^{-3} = 1/2^3 = 1/8$.

Geometric Sums

Suppose we have a loop for which each iteration takes a multiplicative factor longer than the previous one. This loop can be analyzed using the following proposition.

Proposition 4.5: *For any integer $n \geq 0$ and any real number a such that $a > 0$ and $a \neq 1$, consider the summation*

$$\sum_{i=0}^n a^i = 1 + a + a^2 + \cdots + a^n$$

(remembering that $a^0 = 1$ if $a > 0$). This summation is equal to

$$\frac{a^{n+1} - 1}{a - 1}.$$

Summations as shown in Proposition 4.5 are called **geometric** summations, because each term is geometrically larger than the previous one if $a > 1$. For example, everyone working in computing should know that

$$1 + 2 + 4 + 8 + \cdots + 2^{n-1} = 2^n - 1,$$

for this is the largest unsigned integer that can be represented in binary notation using n bits.

4.2.1 Comparing Growth Rates

To sum up, Table 4.2 shows, in order, each of the seven common functions used in algorithm analysis.

constant	logarithm	linear	n -log- n	quadratic	cubic	exponential
1	$\log n$	n	$n \log n$	n^2	n^3	a^n

Table 4.2: Seven functions commonly used in the analysis of algorithms. We recall that $\log n = \log_2 n$. Also, we denote with a a constant greater than 1.

Ideally, we would like data structure operations to run in times proportional to the constant or logarithm function, and we would like our algorithms to run in linear or n -log- n time. Algorithms with quadratic or cubic running times are less practical, and algorithms with exponential running times are infeasible for all but the smallest sized inputs. Plots of the seven functions are shown in Figure 4.4.

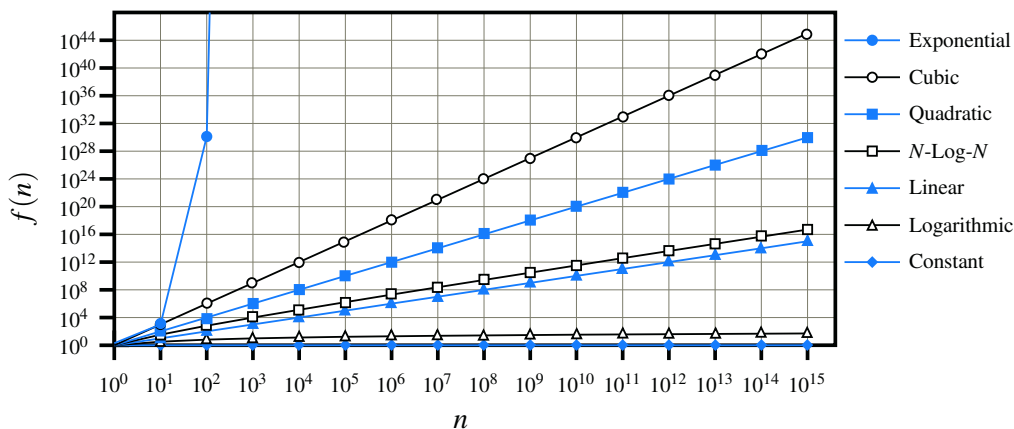


Figure 4.4: Growth rates for the seven fundamental functions used in algorithm analysis. We use base $a = 2$ for the exponential function. The functions are plotted on a log-log chart to compare the growth rates primarily as slopes. Even so, the exponential function grows too fast to display all its values on the chart.

The Ceiling and Floor Functions

When discussing logarithms, we noted that the value is generally not an integer, yet the running time of an algorithm is usually expressed by means of an integer quantity, such as the number of operations performed. Thus, the analysis of an algorithm may sometimes involve the use of the *floor function* and *ceiling function*, which are defined respectively as follows:

- $\lfloor x \rfloor$ = the largest integer less than or equal to x . (e.g., $\lfloor 3.7 \rfloor = 3$.)
- $\lceil x \rceil$ = the smallest integer greater than or equal to x . (e.g., $\lceil 5.2 \rceil = 6$.)